AFRINEXT · PHASE 0 · FOR SENIOR DEVELOPER REVIEW

Phase 0 Review Packet

The production foundation: monorepo, database, authentication, scoped permissions, versioned consent, audit log, double-entry ledger and the payment abstraction. What was executed, what was not, and what is deliberately absent.

Branch **claude/create-app-repository-1cxsih** | Commit **87743b6** | 139 files · +14 468 / −7 049

Tests **102 passed, 3 skipped** | Lint **clean** | Build **green** | iPayMoney **not integrated**

1 Status at a glance

HOW TO READ THIS PACKET

Three states are kept strictly apart throughout. **Tested** means a test was executed and its result is reproduced below. **Not tested** means the code exists but no automated test covers it. **Not implemented** means the code does not exist. Nothing is called "verified" unless the relevant test actually ran.

PHASE 0 ITEM	STATE	EVIDENCE
1 · Monorepo foundation	TESTED	pnpm workspace, 5 packages + 1 app; lint, typecheck, test, build all run across it
2 · Application structure	TESTED	eslint boundary rule forbids `packages/*` importing Next.js; build green
3 · PostgreSQL	TESTED	PostgreSQL 16.13; integration tests run against a real database
4 · Database migrations	TESTED	2 migrations applied; 22 tables; schema rebuilt from scratch in CI
5 · ORM setup	TESTED	Drizzle; every integration test goes through it
6 · Authentication foundation	PARTLY TESTED	Domain core tested (23 tests). **No HTTP routes, no sign-in screen.**
7 · Phone OTP architecture	PARTLY TESTED	Generation, hashing, expiry, attempts, binding tested. **No SMS provider.**
8 · Email authentication architecture	PARTLY TESTED	scrypt hashing and verification tested. **No email sending.**
9 · Secure sessions	PARTLY TESTED	Token issuance, hashing, expiry, revocation, elevation tested. **No cookie wiring.**
10 · Scoped RBAC	TESTED	11 tests including scope confinement and revocation
11 · Permission system	TESTED	22 permissions, 7 roles, seeded and asserted
12 · i18n foundation	NOT TESTED	fr/en catalogues and resolver exist. **No route-level wiring, no tests.**
13 · Design system foundation	NOT TESTED	Tokens extracted to `packages/ui`; app consumes them. No component tests.
14 · CI	TESTED	Executed on GitHub and passed. Runs 32243141943 and 32243758387, all 15 steps green. *Corrected — see below.*
15 · Lint	TESTED	Exit 0 across all 6 projects
16 · Type checking	TESTED	Exit 0 across all 6 projects, strict mode
17 · Automated test framework	TESTED	Vitest + fast-check; 105 specs
18 · Database test environment	TESTED	Separate `TEST_DATABASE_URL`; suite refuses to run without it
19 · Environment configuration	NOT TESTED	`.env.example` committed, `.env` ignored
20 · Secrets handling	TESTED	Redaction by key name tested; no secrets in the repository
21 · Observability / logging	TESTED	5 tests incl. redaction, levels, child context

PHASE 0 ITEM	STATE	EVIDENCE
22 · Audit-log foundation	PARTLY TESTED	Table and writer exist; append-only enforcement tested. **Writer itself has no direct test.**
23 · Initial financial schema	TESTED	14 ledger tests incl. property-based sequences
24 · Payment-provider abstraction	TESTED	13 tests; mock refuses to load in production
25 · iPayMoney adapter architecture	NOT IMPLEMENTED	Boundary throws; 3 sandbox specs skipped. Blocked on documentation.

2 Verify it yourself

Needs Node 22+, pnpm 10+, and a PostgreSQL 16 you can create two databases in. No credentials, no third-party accounts, no network beyond the npm registry.

1 Check out the commit

```
git clone https://github.com/abdoulasani/Afrinext.git
cd Afrinext
git checkout 87743b6
pnpm install
```

2 Point it at a database

`TEST_DATABASE_URL` must differ from `DATABASE_URL`. The suite truncates every table it touches and refuses to start otherwise.

```
createdb afrinext && createdb afrinext_test

cp .env.example .env
# then set, for a local server on the default port:
#   DATABASE_URL=postgres://postgres@127.0.0.1:5432/afrinext
#   TEST_DATABASE_URL=postgres://postgres@127.0.0.1:5432/afrinext_test
```

3 Migrate and seed

```
pnpm db:migrate
pnpm --filter @afrinext/core seed
```

Expected: `Migrations complete.` then `Seeded 7 currencies, 11 countries, 22 permissions, 7 roles, 9 settings, 7 legal documents (placeholder texts).`

4 Run the whole verification chain

```
pnpm lint
pnpm typecheck
pnpm test
pnpm build
```

All four must exit 0. Actual output is reproduced in section 3.

5 Prove the ledger balances over a real cycle

Posts a capture, a provider settlement, a sale settlement with a referral, a payout approval and a payout, then reconciles. This is the gate CI runs.

```
pnpm --filter @afrinext/core exec tsx scripts/reconcile-check.ts
```

6 Check the app still runs, and reaches the database

```
pnpm build && pnpm start &
curl -s localhost:3000/api/health
```

7 Try to break the ledger by hand

Worth doing yourself rather than trusting the tests. Each of these must be refused by the database, not by application code:

```
-- append-only
update ledger_entries set amount_minor = 1;      -- ERROR: append-only
delete from ledger_entries;                      -- ERROR: append-only

-- unbalanced transaction, rejected at COMMIT
begin;
insert into ledger_transactions (id, kind) values (gen_random_uuid(), 'adjustment');
-- ... insert a single entry against that transaction ...
commit;                                          -- ERROR: does not balance
```

3 Results, as executed

> **CORRECTION TO THE FIRST ISSUE OF THIS PACKET**
>
> The first issue of this packet stated that **CI had never executed**, and listed that as a known limitation and an open risk. That was wrong. The workflow had already run on GitHub twice and passed both times — runs 32243141943 (commit `87743b6`) and 32243758387 (commit `ee04b8f`) — before the packet was written. I asserted the negative without checking. The rows above and the risk register have been corrected; the real gap is dependency scanning, which CI does not do.

Copied from the run on commit `87743b6` against PostgreSQL 16.13. Nothing here is predicted.

Tests — `102 PASSED · 3 SKIPPED`

```
✓ src/ledger/ledger.test.ts               (14 tests)  4381ms
✓ src/auth/auth.test.ts                   (23 tests)  1978ms
✓ src/authz/authorize.test.ts             (11 tests)   902ms
✓ src/payments/payments.test.ts           (13 tests)    11ms
✓ src/money/allocate.test.ts               (9 tests)   184ms
✓ src/consent/consent.test.ts              (6 tests)   464ms
✓ src/money/money.test.ts                 (13 tests)   128ms
✓ src/observability/logger.test.ts         (5 tests)     7ms
✓ src/money/currency.test.ts               (7 tests)    10ms
✓ src/payments/ipaymoney.integration.test.ts
                            (4 tests | 3 skipped)     3ms

Test Files  10 passed (10)
     Tests  102 passed | 3 skipped (105)
```

The 3 skipped specs are the iPayMoney sandbox tests. They skip unless `IPAYMONEY_BASE_URL` and `IPAYMONEY_API_KEY` are set, so CI stays green without credentials and turns red the moment a real sandbox run regresses.

Lint and typecheck — `EXIT 0`

```
packages/db    lint: Done        packages/db    typecheck: Done
packages/i18n  lint: Done        packages/i18n  typecheck: Done
packages/ui    lint: Done        packages/ui    typecheck: Done
packages/core  lint: Done        packages/core  typecheck: Done
apps/web       lint: Done        apps/web       typecheck: Done
```

TypeScript runs with `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, `noImplicitOverride`, `noUnusedLocals` and `verbatimModuleSyntax` — set from the first commit, because retrofitting them into a large codebase is miserable.

Build — GREEN

```
✓ Compiled successfully
  Finished TypeScript in 3.0s

Route (app)
┌ ƒ /                          ├ ƒ /browse
├ ○ /_not-found                ├ ƒ /listing/[id]
├ ƒ /api/health                ├ ○ /offline
├ ƒ /api/listings              ├ ○ /saved
├ ƒ /api/listings/[id]         └ ○ /submit
```

Ledger reconciliation gate — CLEAN

A full capture → provider settlement → sale settlement → payout approval → payout, then reconciliation:

```
{
  "scenario": {
    "gross": "10000", "seller": "8200",
    "platform": "1620", "referral": "180"
  },
  "checkedAccounts": 8,
  "drift": 0,
  "unbalancedTransactions": 0,
  "netByCurrency": { "XOF": "0" }
}
Ledger reconciliation clean over a real capture → settlement → payout cycle.
```

8 200 + 1 620 + 180 = 10 000 exactly. The referral is 10% of Afrinext's 1 800 commission, not of the gross, so it can never exceed what was earned on the sale.

Health endpoint — REACHABLE

```
$ curl -s localhost:3000/api/health
{ "status": "ok", "database": "reachable",
  "migrationsApplied": 2, "currenciesSeeded": 7,
  "permissionsSeeded": 22, "paymentProvider": "mock", "latencyMs": 2 }
```

This is the end-to-end proof that the monorepo wiring works: the Next.js route calls `packages/core`, which calls `packages/db`, which reaches PostgreSQL — and the structured logger emits it.

GitHub CI — EXECUTED AND GREEN

Run 32243758387 on commit `ee04b8f`, 79 seconds, ubuntu-latest with a `postgres:16` service. Every step succeeded:

```
✓ Initialize containers            ✓ Run pnpm lint
✓ actions/checkout@v4              ✓ Run pnpm typecheck
✓ pnpm/action-setup@v4             ✓ Run pnpm test
✓ actions/setup-node@v4            ✓ Build the web app
✓ pnpm install --frozen-lockfile   ✓ Assert the ledger reconciles after the suite
✓ Create the test database
✓ Migrate
✓ Rebuild the schema from scratch
✓ Seed reference data
```

This is the distinction the review asked for: everything above is **GitHub CI verification**, not local verification. The two agree.

> **ON SCREENSHOTS**
>
> None are included, deliberately. Phase 0 shipped **no new user-facing screens** — the app served is still the original directory prototype. A screenshot of it would suggest progress that did not happen. The terminal output above is the evidence that matters here.

4 Architecture changes

The blueprint is now revision 2. Four things changed, all of them reflecting confirmed business decisions rather than engineering opinion.

CHANGE	WHERE
New section 01 — company, contract and regulatory posture. Records AFRI NEXT TECHNOLOGIE as operator, and separates seven layers that get conflated: company registration (A), contract with users (B), payment-provider relationship (C), internal ledger (D), seller entitlement (E), payout operation (F) and regulatory authorization (G). States plainly that NIF and RCCM establish A only, and that nothing in A–F establishes G.	`blueprint §01`
Consent architecture. Versioned documents, append-only acceptance records, and a server-side gate. Consent establishes B, never G.	`blueprint §01`
Confirmations. Referral single-level, V1/V2/V3 scope, Niger + XOF as launch market — all moved from "recommendation" to "confirmed". V2 and V3 architecture retained deliberately.	`§00, §H, §M`
iPayMoney section rewritten. Confirmed as provider with a merchant account; API unverified and unreachable; 13 documentation items listed.	`§P`

> **THE CONSEQUENCE IN CODE**
>
> Because layer G is unresolved, three things are true of the implementation and must stay true: **the ledger records entitlement, not custody** — nothing asserts where money physically sits; **the payout rail is an interface**, so the answer can change without a rewrite; and **no field or screen calls a seller balance a deposit or protected funds**.

5 Database schema and migrations

22 tables, 2 migrations, 6 triggers, 1 view. Migration `0000` is generated from the Drizzle schema; `0001` is hand-written and is the one worth reading closely.

GROUP	TABLES
Reference	`currencies, countries, locales`
Identity	`users, user_identities, sessions, otp_challenges, rate_limit_counters`
Authorization	`permissions, roles, role_permissions, role_assignments`
Consent	`legal_documents, legal_document_versions, consent_records`
Ledger	`ledger_accounts, ledger_transactions, ledger_entries, account_balances, idempotency_keys`
Operations	`audit_logs, platform_settings`

What migration 0001 adds, and why it is not in application code

GUARANTEE	MECHANISM	TESTED
Every transaction balances, per currency	`CONSTRAINT TRIGGER … DEFERRABLE INITIALLY DEFERRED` — evaluated at COMMIT, so all entries can be inserted first	YES
An entry's currency matches its account	`BEFORE INSERT` trigger	YES
Ledger entries and transactions are append-only	`BEFORE UPDATE OR DELETE` trigger that raises	YES
Consent records and audit logs are append-only	Same	YES (CONSENT)
Least-privilege application role	`afrinext_app` role with UPDATE/DELETE revoked on the immutable tables	NOT TESTED — dev connects as superuser
Authoritative balance for reconciliation	`ledger_account_balances_derived` view	YES

A trigger is used rather than only a `REVOKE` because a superuser ignores `REVOKE`, and development connections often are superuser. The revoke is defence in depth behind it.

> **WORTH YOUR ATTENTION**
>
> Drizzle has no down-migrations. CI compensates by dropping the schema and rebuilding it from zero on every run, which proves the migrations apply cleanly to an empty database — but it is **not** the same as being able to roll back a release in production. If you want true reversibility, say so and it becomes a Phase 1 task.

6 Authentication status

READ THIS FIRST

Nobody can sign in yet. Phase 0 built the authentication *domain core* — the primitives, tested in isolation. There are no HTTP routes, no cookie handling, no SMS or email delivery, and no screens. Wiring lands in Phase 1.

PIECE	STATE	DETAIL
Password hashing	TESTED	scrypt ($N=2^{16}$, r=8, p=1). Encoded hash carries its own parameters, so the algorithm can change without invalidating existing hashes. `needsRehash()` flags weak ones.
Session tokens	TESTED	32 random bytes; only the SHA-256 is stored, so a database leak does not hand over live sessions. Constant-time comparison.
Session lifecycle	TESTED	Expiry, revocation, and a 10-minute elevation window for payout-sensitive actions.
OTP	TESTED	Cryptographically uniform 6 digits, hashed with identifier and purpose as salt, bounded attempts, 5-minute expiry. A code issued for sign-in will not verify for step-up, or for a different phone number.
Phone normalisation	TESTED	E.164 only, calling codes from the `countries` table, longest-prefix match. Property test: any input either yields valid E.164 or an explicit failure.
OTP rate limiting	NOT IMPLEMENTED	The `rate_limit_counters` table exists; nothing writes to it yet. **Bounding attempts alone does not stop an attacker who can request unlimited fresh codes** — this is a Phase 1 must.
SMS / email delivery	NOT IMPLEMENTED	No provider chosen. The OTP core is provider-agnostic.
HTTP routes, cookies, screens	NOT IMPLEMENTED	Phase 1.

7 Authorization status

Scoped role assignments, one gate, 22 permissions across 7 roles. `11 TESTS`

There is **no `role` column on `users`**. Roles are rows in `role_assignments` carrying a scope type and id, so one person being a buyer, an instructor, a rider and staff on someone else's store at the same time costs nothing.

BEHAVIOUR	TESTED
A user with no roles is denied everything	YES
A scoped role does not leak into another scope — store owner of A cannot publish to store B	YES
A global grant satisfies a scoped check	YES
A revoked assignment stops being honoured	YES
Permissions accumulate across several roles	YES
Nobody can approve their own withdrawal, whatever roles they hold	YES
`superadmin` deliberately lacks `withdrawal.approve`	YES
`support` and `ops` hold no financial capability	YES

Administering roles and moving money are separate capabilities by design: `superadmin` can grant roles but cannot approve a payout, and `finance` can approve payouts but cannot change who holds which role. That split is only meaningful if nobody is granted both — an operational rule the console will need to surface.

8 The ledger

The part worth most of your review time. `14 LEDGER TESTS · 29 MONEY TESTS`

Money

`Money` is `{ amountMinor: bigint, currency: string }`. No floats anywhere. The minor-unit exponent is read from the `currencies` table, never assumed — **XOF and XAF have zero decimal places**, so code that divides by 100 is wrong by a factor of 100 across the whole UEMOA zone. A test asserts this directly.

The split rule

Every non-residual line is rounded half-up; the seller's share is `gross − Σ(other lines)`, never computed independently. So a split always re-sums to the gross and the remainder consistently favours the seller.

Referral commission is drawn **from Afrinext's own commission**, not from the gross. Property tests over 2 000 randomised cases assert:

- the split re-sums to the gross exactly, whatever the rates;
- no share is ever negative;
- **the referral never exceeds what the platform earned** — the confirmed guardrail;
- the rounding remainder lands with the seller, not the platform.

Invariants proved by test

INVARIANT	HOW IT IS PROVED
Every transaction balances per currency	Deferred DB trigger; a hand-crafted unbalanced insert is rejected at COMMIT
Entries are immutable	UPDATE and DELETE both raise; asserted directly
The whole ledger nets to zero per currency	Asserted after every operation in the property test
Cached balance equals the sum of entries	Asserted after every operation; drift injected by hand is detected
Duplicate operations are harmless	Same idempotency key replays and returns the original transaction
A key reused with different inputs is refused	Throws `IdempotencyConflictError` rather than replaying someone else's result
Reversals restore state without editing history	Balances return to zero; the original entries still exist alongside the mirror
Money never disappears on a failed payout	Funds return to the user's available balance

The property test

25 randomised runs of 4–14 operations each — captures, settlements with and without referral, hold releases, payout approvals and settlements, and reversals — across three sellers and an ambassador.

After every single operation it asserts: no unbalanced transaction exists, the ledger nets to zero per currency, and every cached balance equals its derived balance.

The test distinguishes a deliberate domain refusal from a broken invariant: a refused operation is allowed, anything else re-throws and fails the property.

> **WHAT THE LEDGER TESTS DO NOT COVER**
>
> - **Concurrency.** Every test is single-threaded. The balance cache uses an atomic increment under the row lock taken by `ON CONFLICT DO UPDATE`, which should serialise concurrent posts — but that reasoning is **not tested**. A concurrent-posting test belongs in Phase 1.
> - **Multi-currency transactions.** Every test uses XOF. The schema and code handle several currencies per transaction; nothing exercises it.
> - **Scale.** Largest run is a few hundred entries. No behaviour is known at millions of rows.

9 Payments

> **IPAYMONEY IS NOT INTEGRATED**
>
> Confirmed as the provider, and Afrinext holds a merchant account — but the official documentation is not in the repository and the provider's domain is **not reachable from the build environment** (egress blocked by policy). So nothing about the API has been verified: not the authentication scheme, not an endpoint, not a payload, not a webhook signature.
>
> `IPayMoneyProvider` exists as a boundary and **every method throws** `ProviderNotConfiguredError`, naming what is missing. A stub returning plausible data would let the system appear to work and fail in production, on money.

PIECE	STATE
`PaymentProvider` interface	TESTED Provider-agnostic. `createPayout` is optional because whether iPayMoney offers disbursement is unknown; `supportsPayouts()` is how callers check.
`MockPaymentProvider`	TESTED Charges, idempotency, refunds, payouts, and HMAC webhook verification against the raw body with constant-time comparison. Tampered bodies and bad signatures both rejected.
Production safety	TESTED The mock throws if constructed when `NODE_ENV=production`, and the registry refuses to hand it out. A misconfigured environment stops the process rather than quietly accepting payments that never happened.
Unknown provider id	TESTED Refused rather than silently falling back
iPayMoney adapter	NOT IMPLEMENTED Boundary only
Sandbox integration specs	SKIPPED 3 specs, gated on credentials in the environment

The single most useful thing you can send is iPayMoney's official documentation — the 13 items are listed in `docs/providers/ipaymoney/README.md`. Items 6 (webhook signature), 7 (whether XOF is sent as whole francs) and 8 (whether a payout API exists) most often invalidate an assumed design.

10 Security checks

CONTROL	STATE	NOTE
Passwords never stored in the clear	TESTED	scrypt with per-hash salt; parameters encoded in the hash
Session tokens never stored in the clear	TESTED	SHA-256 only; constant-time comparison
OTP codes never stored in the clear	TESTED	Salted by identifier and purpose, so one rainbow table of six-digit strings does not work
Secrets never reach the logs	TESTED	Redaction by key name, case- and punctuation-insensitive, recursive
No secrets in the repository	TESTED	`.env` git-ignored; `.env.example` holds no values
Separation of duty on payouts	TESTED	Self-approval blocked in code
IDOR prevention	PARTLY	Authorization scopes queries in SQL rather than checking after fetch. **Untested at the HTTP layer, because there are no protected routes yet.**
Webhook signature verification	MOCK ONLY	The pattern is correct — verify the raw body before parsing, constant time. The real scheme is unknown.
Idempotency on money operations	TESTED	Request hash stored; key reuse with different inputs refused
Audit log	PARTLY	Table and writer exist and the table is append-only. **No call sites yet** — nothing is being audited in practice.
Rate limiting	NOT IMPLEMENTED	Table exists, nothing writes to it
CSRF, security headers, CSP	NOT IMPLEMENTED	No authenticated routes yet to protect
Dependency vulnerability scanning	NOT IMPLEMENTED	Not in CI. Worth adding in Phase 1.
Penetration test	NOT DONE	Schedule before public launch, not after

STILL TRUE FROM PHASE 1, AND UNCHANGED

The prototype's `POST /api/listings` remains open to anyone — it predates authentication and is scheduled for replacement. It writes only to an in-memory store, so nothing persists, but it should not survive into anything public.

11 What is NOT implemented

Complete list. Anything not here either works or is named as untested elsewhere in this packet.

AREA	NOT IMPLEMENTED
Authentication	Sign-in, sign-up and sign-out routes; cookie handling; SMS and email delivery; OTP issuance rate limiting; any screen. **Nobody can sign in.**
Payments	The iPayMoney adapter. Blocked on documentation.
Commerce	Stores, products, courses, cart, checkout, orders, fee-line snapshots. No commerce tables exist yet — deliberately, they belong to Phase 1.
Wallet and payouts	Withdrawal requests, KYC, payout batches, the finance console. The ledger movements exist; the workflow around them does not.
Referral	Attribution, qualification, commission records. The split arithmetic exists and is tested; the programme is not built.
Learn	Courses, lessons, entitlement, video, progress.
Delivery	Everything. V2.
Admin console	No screens. Each phase ships its own admin surface.
i18n	Route-level integration (`/fr/…` , `/en/…`) and the provider wiring. Catalogues and the resolver exist; nothing consumes them yet.
Legal document texts	Placeholders only, with content hashes recording them as such. The machinery works with any text; drafting is a legal deliverable.
Audit call sites	The writer exists but nothing calls it. Nothing is being audited yet.
Turborepo	Not added. Plain pnpm scripts are sufficient at 6 projects; revisit when build times justify it.
Deployment	No hosting, no environments, no deploy pipeline. Awaiting the hosting decision.

Known limitations of what *is* built

- **Dependency vulnerability scanning is absent from CI.** The workflow runs lint, typecheck, tests, a schema rebuild and the reconciliation gate, but nothing scans dependencies. Added in Phase 1.
- **No concurrency testing** anywhere, including the ledger's balance cache.
- **No down-migrations.** CI rebuilds from scratch instead; that is not the same as rolling back a release.
- **The least-privilege role is untested** — local development connects as superuser, so the `REVOKE` s are not exercised. The append-only triggers are, and they bind superusers too.
- **Single currency in practice.** All tests use XOF.
- **The web app is still the prototype.** Its in-memory store is untouched by Phase 0 and still resets on restart.
- **Latency assumption unverified.** `eu-west-3` over `af-south-1` is still routing intuition, not measurement.

12 Deviations from the blueprint — your call

Four places where the implementation differs from what the blueprint proposed. Each was a judgement call made to keep Phase 0 moving; each is cheap to reverse now and expensive later.

Deviation 1

Auth core written in-house instead of adopting Better Auth

Why
: What Phase 0 needed was session lifecycle, OTP lifecycle and password hashing — roughly 300 lines of framework-free, fully testable code with no dependency to keep patched. Better Auth brings its own session tables that would have to be reconciled with the scoped RBAC and audit design, and its phone-OTP path cannot be exercised without an SMS provider we have not chosen.

What was not written
: No cryptography was invented. scrypt and SHA-256 come from `node:crypto`; randomness from `randomBytes` and `randomInt`.

Risk
: Rolling your own auth is a classic mistake. The mitigation is that this is session *management*, not cryptography, and it is behind an interface — adopting Better Auth later replaces one module.

Your call
: Accept, or direct me to adopt Better Auth in Phase 1 before the HTTP layer is built on top.

Deviation 2

scrypt rather than Argon2id

Why
: OWASP accepts both. scrypt is in `node:crypto` with no native binding to build or audit, and phone OTP — not a password — is the primary credential in this market.

Cost of changing
: Low. The encoded hash carries its own parameters, so existing hashes keep verifying while new ones use Argon2id, and `needsRehash()` already flags the old ones.

Note
: Parameters are N=2^{16} (~64 MiB per hash) rather than OWASP's suggested 2^{17}. 2^{17} doubles the memory per login and with concurrency becomes a denial-of-service surface. Say the word and it moves.

Deviation 3

Drizzle adopted while the ORM question is still open

Why
: Phase 0 could not start without an ORM, and the blueprint's stated recommendation was Drizzle. The decision genuinely depends on team composition, which is still unanswered.

Cost of changing now
: Moderate: the schema definitions and roughly a dozen query sites. The SQL, the migrations and every invariant in migration 0001 are ORM-independent and would carry over unchanged.

Cost of changing after Phase 1
: Considerably higher.

Your call
: Tell me the team's Prisma versus raw-SQL experience and I will either confirm or switch — now, not later.

Deviation 4

i18n foundation without route-level wiring

Why
: Wiring `/fr/…` and `/en/…` means restructuring every route — and every current route is prototype code scheduled for replacement. Doing it now would be churn against pages that are about to be deleted.

What exists
: fr and en catalogues, locale resolution, placeholder interpolation, and the `locales` table. French is the default.

Consequence
: Phase 1's first real screens must land with the route wiring, not after it. Flagged so it does not slip.

13 Unresolved risks

RISK	SEV	WHERE IT STANDS
Regulatory authorization (layer G). Afrinext receiving customer money and paying sellers may be a regulated activity under BCEAO rules. NIF and RCCM establish company registration only.	HIGH	Unresolved and outside engineering. The architecture reduces exposure the only way it can — the ledger records entitlement rather than asserting custody, and the payout rail is an interface. **Gates Phase 6, not Phase 1.** Long lead time; worth starting now.
iPayMoney API unknown. Documentation absent and the domain unreachable from the build environment.	HIGH	Contained: the abstraction is built and the mock exercises the whole cycle. Nothing else is blocked. 13 items listed in `docs/providers/ipaymoney/README.md`.
CI has no dependency scanning. The workflow itself is proven — it ran on GitHub and passed twice.	MED	Scanning added in Phase 1.
Ledger concurrency untested. The balance cache relies on row-lock serialisation that has been reasoned about, not tested.	MED	Add a concurrent-posting test in Phase 1, before real traffic exists.
Nothing is audited yet. The writer exists but has no call sites.	MED	Must land with the first financial and administrative actions in Phase 1, not after.
OTP issuance is unlimited. Attempts are bounded; requests are not.	MED	Each SMS costs money, so this is a billing risk as well as a security one. Phase 1 must.
No rollback path for migrations.	MED	CI rebuilds from scratch. Decide whether true down-migrations are required.
Legal texts do not exist. Consent machinery works; the documents are placeholders.	MED	Blocks nothing technical. Blocks going live.
No dependency scanning or penetration test.	MED	Add scanning to CI in Phase 1; schedule the test before public launch.
Latency assumption unverified.	LOW	Measure from Niamey before committing to a region.

14 Next phase proposal

> NOT STARTED
>
> Phase 1 has not begun and will not begin without your explicit approval. This is a proposal.

Phase 1 — identity in use, and the commerce spine · ~3 weeks

Phase 0 built primitives nobody can reach. Phase 1 makes them reachable and puts the first real data behind them.

1. **Authentication over HTTP.** Sign-in, sign-up, sign-out and step-up routes; secure cookie handling; an SMS provider behind an interface; **OTP issuance rate limiting**; the first real screens, with i18n routing wired in from the start.
2. **Audit call sites.** Every authentication event, role change and ledger post writes an audit row in the same transaction.
3. **Consent in the flow.** The gate applied at sign-up and seller onboarding, with real (or at least drafted) document texts.
4. **Stores and the catalogue.** Store creation, digital products and courses as products, categories, media upload.
5. **Commerce tables.** Cart, orders, order items and `order_fee_lines` — the frozen commission snapshot, so a rate change never rewrites history.
6. **Concurrency test** for the ledger, and a dependency scan in CI.

Exit criterion: a real person registers by phone, accepts the terms, creates a store, publishes a paid digital product, and it appears at a public URL — with every step audited and the ledger untouched because nothing has been bought yet.

Payment integration stays gated on the iPayMoney documentation. If it has not arrived by the end of Phase 1, the manual transfer rail from the roadmap's P3 becomes the launch payment method and we should plan for that openly rather than waiting.

15 Senior developer sign-off checklist

Suggested order. The first four are where a wrong answer is most expensive to discover later.

#	CHECK	WHERE	VERDICT
1	Are the ledger invariants the right set, and is a deferred constraint trigger the right enforcement point? Anything missing?	`packages/db/migrations/0001_ledger_invariants.sql`	☐ approve ☐ changes ☐ reject
2	Does the property test actually prove what this packet claims? Read it adversarially — could it pass while the ledger is broken?	`packages/core/src/ledger/ledger.test.ts`	☐ approve ☐ changes ☐ reject
3	Is the residual rule sound, and is drawing referral from platform commission implemented as intended?	`packages/core/src/money/allocate.ts`	☐ approve ☐ changes ☐ reject
4	Is the posting path correct — idempotency, balance-cache increment, transaction boundaries?	`packages/core/src/ledger/post.ts`	☐ approve ☐ changes ☐ reject
5	Is `authorize()` a sufficient gate, and is scope confinement right?	`packages/core/src/authz/authorize.ts`	☐ approve ☐ changes ☐ reject
6	Is the in-house auth core acceptable, or should Better Auth be adopted before the HTTP layer? **(Deviation 1)**	`packages/core/src/auth/`	☐ approve ☐ changes ☐ reject
7	scrypt at $N=2^{16}$, or move to Argon2id? **(Deviation 2)**	`packages/core/src/auth/password.ts`	☐ approve ☐ changes ☐ reject
8	Confirm or switch the ORM — decide now, not after Phase 1. **(Deviation 3)**	`packages/db/src/schema/`	☐ confirm Drizzle ☐ switch to Prisma
9	Is the schema right? Anything missing that Phase 1 will need, or present that it will not?	`packages/db/src/schema/`	☐ approve ☐ changes ☐ reject
10	Is the payment abstraction sound, and is throwing the right behaviour for an unverified provider?	`packages/core/src/payments/`	☐ approve ☐ changes ☐ reject
11	Is the framework boundary drawn in the right place, and worth enforcing by lint?	`packages/config/eslint.base.mjs`	☐ approve ☐ changes ☐ reject

#	CHECK	WHERE	VERDICT
12	Does the CI workflow cover what it should? It runs green on GitHub; dependency scanning is the known gap.	`.github/workflows/ci.yml`	☐ approve ☐ changes ☐ reject
13	Are down-migrations required before production?	`packages/db/migrations/`	☐ required ☐ not required
14	Is the Phase 1 proposal the right next step, and its exit criterion the right bar?	`section 14`	☐ approve ☐ changes ☐ reject

ALSO NEEDED, AND NOT FROM ENGINEERING

- **iPayMoney documentation** — the 13 items. Sandbox credentials are the most useful single thing.
- **Who is advising on layer G** (regulatory authorization), and what is being pursued.
- **Team composition** — settles checklist item 8.
- **Hosting** — managed or containers.
- **SMS provider for Niger** — needed before OTP can actually be delivered in Phase 1.

Phase 0 complete · branch `claude/create-app-repository-1cxsih` · commit `87743b6` · Phase 1 will not begin without explicit approval.

Afrinext Technical Blueprint

The architecture for a multi-country African commerce and learning platform built around a double-entry ledger — stack, schema, permission model, money design, security controls and a phased build order.

Rev 2 — Phase 0 authorised | Operator: AFRI NEXT TECHNOLOGIE | Launch: Niger · XOF | iPayMoney: not integrated

CONTENTS

00 The read, and one pushback

Afrinext is three businesses sharing one identity system. A **marketplace** (physical goods, digital goods, services), a **learning platform** (paid video and documents with entitlement), and a **money system** (wallet, commissions, settlement, payouts). Delivery and the referral network are not separate products — they are two more sources of ledger events.

That framing decides the architecture. The ledger is the spine; everything else is a producer of financial events that the ledger records. A course purchase, a delivered parcel and an ambassador commission are the same shape of thing once they reach the money layer. If that layer is designed last, every module above it grows its own private notion of "balance" and the platform becomes unauditable.

The "one person, one account, many roles" principle is right and it is cheap to honour, provided one rule holds from the first migration: **there is no `role` column on the user.** Roles are rows, they are scoped to a resource, and they are additive.

The pushback

The module list in the brief is a two-to-three-year product. Built as specified — all modules in parallel — every part arrives shallow, and the hardest, least reversible part (money) gets exercised last, by which point the schema is load-bearing for a dozen features.

CONFIRMED AS V1

Launch one vertical: digital products and courses, one country, one payment rail.

WHY	It exercises identity, roles, stores, catalog, checkout, payments, ledger, commissions, wallet, withdrawals, entitlement and protected video — every hard part except logistics — with no inventory, no couriers, no cash handling and near-instant fulfilment. It is the shortest path to a real settled transaction, which is the only thing that proves the platform works.
INSTEAD OF	Launching the full marketplace, delivery network and creator marketplace together.
TRADE-OFF	Physical goods and delivery slip to a second launch, roughly five weeks of work that starts later rather than never. Ambassadors have a thinner catalog to promote at day one.
REVERSIBLE?	Yes. Nothing in this blueprint's schema or module boundaries assumes the narrow scope — the tables for physical goods and delivery are designed here, just not built first.

Confirmed. The staged scope is now fixed as below. The V2 and V3 architecture stays in this document even though it is not being built yet — removing it would only mean rediscovering it later.

	SCOPE	STATUS
V1	Identity, authentication, multiple roles, stores, digital products, PDF/ebooks/guides, courses, learning progress, marketplace, checkout, payment integration, ledger, commission engine, seller financial records, payout architecture, Afrinext Network, admin foundation	BUILDING
V2	Physical products, inventory, delivery partners, delivery state machine, proof of delivery	DESIGNED, NOT BUILT
V3	Creator marketplace, advanced services marketplace, further ecosystem functionality	DESIGNED, NOT BUILT

Launch market is **Niger**, launch currency **XOF**. Neither is hard-coded: countries and currencies are rows, and the multi-country design elsewhere in this document is unchanged.

What is in the repository today

The current `Afrinext` repository holds a mobile-first PWA directory prototype: Next.js 16, Tailwind v4, an in-memory listing store, seed data, and a submission form. It is honest about what it is — there is no database, no authentication, no payments, and no money. In this plan:

ASSET	DISPOSITION
Design tokens, PWA shell, mobile navigation, card and form components	**CARRIES FORWARD** Becomes the seed of `packages/ui`.
Next.js 16 + TypeScript + Tailwind v4 setup, lint and build config	**CARRIES FORWARD** Becomes `apps/web` inside the monorepo.
`src/lib/store.ts`, `src/lib/seed.ts`, the in-memory model	**DISCARD** Replaced by Postgres and Drizzle.
`src/lib/search.ts`, `validate.ts`, the URL-driven filter pattern	**REWRITE, KEEP THE SHAPE** The pattern is right; the implementation moves to the database and to shared Zod schemas.

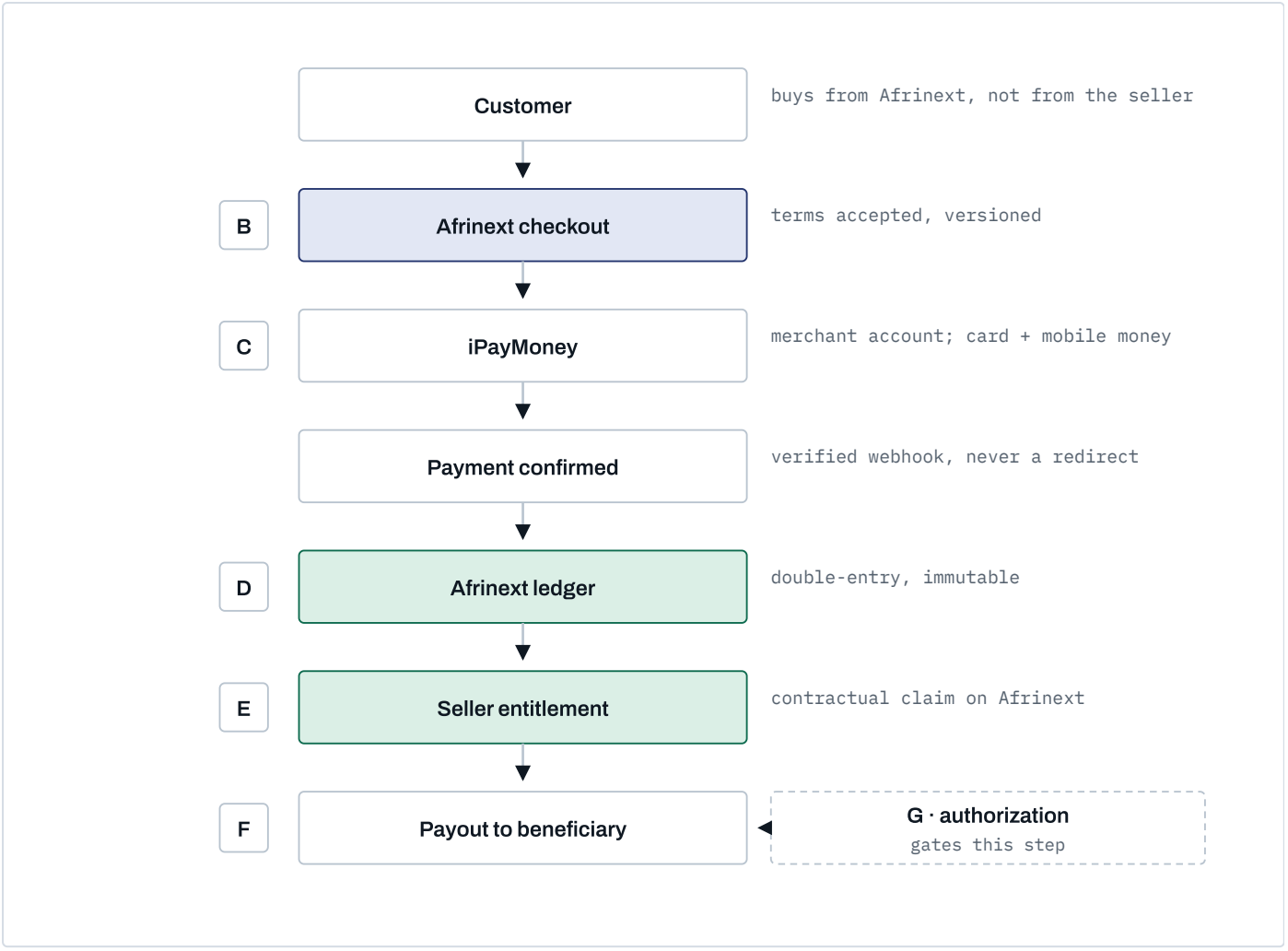
01 Company, contract and regulatory posture

Afrinext is operated by **AFRI NEXT TECHNOLOGIE**, an *Entreprise Individuelle* registered in Niger, holding an NIF and an RCCM. Its registered activities include information technology, artificial intelligence, general commerce, service provision and import-export.

> **WHAT NIF AND RCCM ARE — AND ARE NOT**
>
> They establish that the business exists and is registered to trade: a tax identifier and a commercial-register entry. **They are not a payment-services or e-money licence.** Nothing in this document, in the product, in marketing copy, or in anything said to a user may describe them as one. Company registration and regulatory authorization are different things, granted by different authorities, for different activities.

The confirmed economic model

Afrinext operates as the **commercial platform of record**. The customer contracts with Afrinext, iPayMoney processes the payment, Afrinext's ledger records what each seller, instructor or partner is owed, and Afrinext pays the beneficiary under its own published terms and payout procedure. This is a confirmed business decision, and the architecture is built for it.



Each hop is governed by a different layer, and the layers are not interchangeable. The letters map to the table below. The one that is not yet settled — G — is the one that gates paying money out, which is why the payout rail is behind an interface rather than wired to a single provider.

Seven layers that must never be conflated

	LAYER	ESTABLISHED BY	DOES *NOT* ESTABLISH
A	**Company registration** AFRI NEXT TECHNOLOGIE exists and may trade	NIF, RCCM	Any right to conduct regulated payment activity
B	**Contract with users** Terms binding Afrinext and each user	Versioned acceptance records	Regulatory authorization
C	**Payment-provider relationship** Ability to accept card and mobile-money payments	iPayMoney merchant account	That Afrinext may itself hold or transmit funds as its own business activity
D	**Afrinext internal ledger** Authoritative record of who is owed what	Double-entry entries in Afrinext's database	Any claim on funds physically held anywhere
E	**Seller economic entitlement** A seller's claim against Afrinext	Ledger balance + accepted Seller Terms	A deposit, a bank balance, or protected client money
F	**Payout operation** The act of paying a beneficiary	Payout workflow + a provider or bank rail	Its own lawfulness — see G
G	**Regulatory authorization** Permission to conduct regulated payment activity	A licence, an exemption, or operating under a licensed institution	**Nothing else in this table establishes it.**

Afrinext intends to investigate and obtain whatever additional authorization its financial activities require. Until that is settled, three architectural consequences hold, and they are not negotiable in the code:

1. **The ledger models entitlement, not custody.** A seller's available balance is a contractual claim on Afrinext, and the schema, the API and the interface language all say so. No screen calls it a deposit, an account balance, or protected funds.
2. **The payout rail is an interface, not a provider.** Whatever posture G resolves to — Afrinext licensed, Afrinext operating under a licensed institution, or payouts executed by a third party — the change lands in one adapter, not across the codebase.
3. **Custody is an assumption, not a fact in the code.** Nothing in the ledger asserts where money physically sits. That is deliberate: if the answer changes, the accounting does not.

Consent: versioned, per document, per user

Users accept Afrinext's terms before using the commercial functionality those terms govern. Acceptance is recorded per document *version*, so the question

"which terms was this user bound by when they sold that course in March?" has an answer.

```
CREATE TABLE legal_documents (
  id            uuid PRIMARY KEY,
  kind          text NOT NULL,      -- terms_of_use | privacy_policy | seller_terms |
                                    -- instructor_terms | payment_refund_policy |
                                    -- payout_terms | referral_terms | delivery_terms
  country_code  char(2),            -- NULL = applies everywhere
  UNIQUE (kind, country_code)
);

CREATE TABLE legal_document_versions (
  id             uuid PRIMARY KEY,
  document_id    uuid NOT NULL REFERENCES legal_documents(id),
  version        text NOT NULL,      -- '2026-08-01' or semver; ordered by effective_from
  locale         text NOT NULL,      -- fr | en
  content_hash   text NOT NULL,      -- sha256 of the exact text shown to the user
  effective_from timestamptz NOT NULL,
  UNIQUE (document_id, version, locale)
);

CREATE TABLE consent_records (
  id                  uuid PRIMARY KEY,
  user_id             uuid NOT NULL REFERENCES users(id),
  document_version_id uuid NOT NULL REFERENCES legal_document_versions(id),
  accepted_at         timestamptz NOT NULL DEFAULT now(),
  ip_address          inet,
  user_agent          text,
  locale              text,
  method              text NOT NULL,  -- signup_checkbox | seller_onboarding | reaccept …
  UNIQUE (user_id, document_version_id)
);
```

Enforcement is a gate, not a banner: an action whose terms the user has not accepted at the *current effective version* is refused server-side, and the client is told which document to present. Publishing a new version therefore re-prompts affected users automatically, with the old acceptance preserved rather than overwritten — consent records are append-only, like ledger entries, for the same reason.

> **THE LINE THAT MUST NOT BE CROSSED**
>
> Consent establishes layer B. It does not establish layer G. A user agreeing to Afrinext's Payout Terms does not authorize Afrinext to conduct a regulated activity, and no amount of well-drafted terms substitutes for the competent authority's permission. Anyone who proposes otherwise has misread this section.

A Technology stack

Every choice below is stated with the alternative I rejected, because several of them are genuine trade-offs rather than obvious calls.

LAYER	CHOICE	WHY THIS, OVER THE ALTERNATIVE
Language	TypeScript 5, strict	One language across web, worker and shared domain code. `strict`, `noUncheckedIndexedAccess` and `exactOptionalPropertyTypes` on from day one — retrofitting them into a 40-table codebase is miserable.
Web	Next.js 16, App Router	A marketplace lives or dies on organic search, so product, store and course pages need server rendering and stable URLs. React Server Components also ship less JavaScript to the low-end Android devices that dominate the target market. Rejected: a SPA + separate API, which doubles the deployment surface and loses SEO.
Domain logic	packages/core	Framework-free TypeScript. Next.js is transport, not the application. This is the single most important structural decision in the document — see section L.
Database	PostgreSQL 16+	Not negotiable. Real transactions, `SERIALIZABLE` isolation, CHECK and EXCLUSION constraints, deferred constraint triggers, partial indexes, `NUMERIC`, row-level locking, full-text search. The financial invariants belong in the database, not only in application code.
ORM	Drizzle ORM	See the decision block below — this one is contested.
Background jobs	pg-boss (Postgres-backed)	Lets a job be enqueued *inside the same transaction* as the ledger rows that job refers to. With Redis you get a two-phase failure: the database commits, the enqueue fails, and settlement silently never runs. Rejected: BullMQ + Redis, which is faster but buys throughput we do not have yet with a correctness hole we cannot afford. Revisit above roughly 100 jobs/second.
Auth	Better Auth + DB sessions	Phone-first OTP is a first-class requirement in this market, not an add-on. Database-backed sessions rather than stateless JWTs, because a session that controls a wallet must be revocable instantly. Rejected: Clerk and Auth0 (per-MAU USD pricing against XOF revenue), Lucia (retired upstream).
Validation	Zod	One schema per boundary — HTTP body, server action input, job payload, webhook payload — with types inferred rather than restated.
i18n	next-intl	French as the default locale, English second, locale in the URL path (`/fr/…`) so both index separately. Message catalogs live in `packages/i18n` so the worker can localise SMS and email too.
Styling	Tailwind v4 + tokens	Already in the repository, already token-driven, already dark-mode aware. No reason to change it.
Object storage	Cloudflare R2	Zero egress fees. For a platform whose core product is video in a market where bandwidth is the dominant cost, this is the difference between viable and not. S3-compatible, so the provider interface stays portable.
Video	Cloudflare Stream	Signed playback tokens, HLS adaptive bitrate, per-video access control. We store a `playback_id`, never a file URL. Alternatives worth pricing: Bunny Stream (cheaper), Mux (best tooling, most expensive). Rejected: MP4 files in a bucket — no adaptive bitrate on a 3G connection, and no way to expire access.
SMS / email	provider interface	SMS matters more than email here (OTP, order status, payout confirmations). Concrete provider chosen per country at deploy time; the interface is ours.
Observability	OpenTelemetry + Sentry	Structured logs carrying request id, actor id and ledger transaction id. A money platform without traceable financial events is not operable.

LAYER	CHOICE	WHY THIS, OVER THE ALTERNATIVE
Testing	`Vitest + Playwright`	Property-based unit tests on the ledger and commission engine; Playwright end-to-end on the purchase-to-payout path specifically.
Hosting	`containers, Postgres in eu-west-3`	Paris, not Cape Town: round-trip latency from Niamey, Bamako, Abidjan and Dakar to `eu-west-3` is materially better than to `af-south-1`. Cloudflare in front for edge caching. Vercel works for phases 0–2 but the worker is a long-running process that does not fit its model, so plan for containers.

CONTESTED DECISION

Drizzle ORM rather than Prisma

WHY

Migrations are plain SQL that a human reviews before it touches money. CHECK constraints, deferred constraint triggers and `SELECT … FOR UPDATE` are expressible directly in the schema rather than through escape hatches. No query-engine binary, which matters for cold starts and for self-hosting cheaply.

INSTEAD OF

Prisma, which has better developer experience, better relational query ergonomics and far more mature migration tooling for a 40-table schema.

TRADE-OFF

More SQL written by hand, a smaller ecosystem, and a steeper ramp for developers who have only used Prisma.

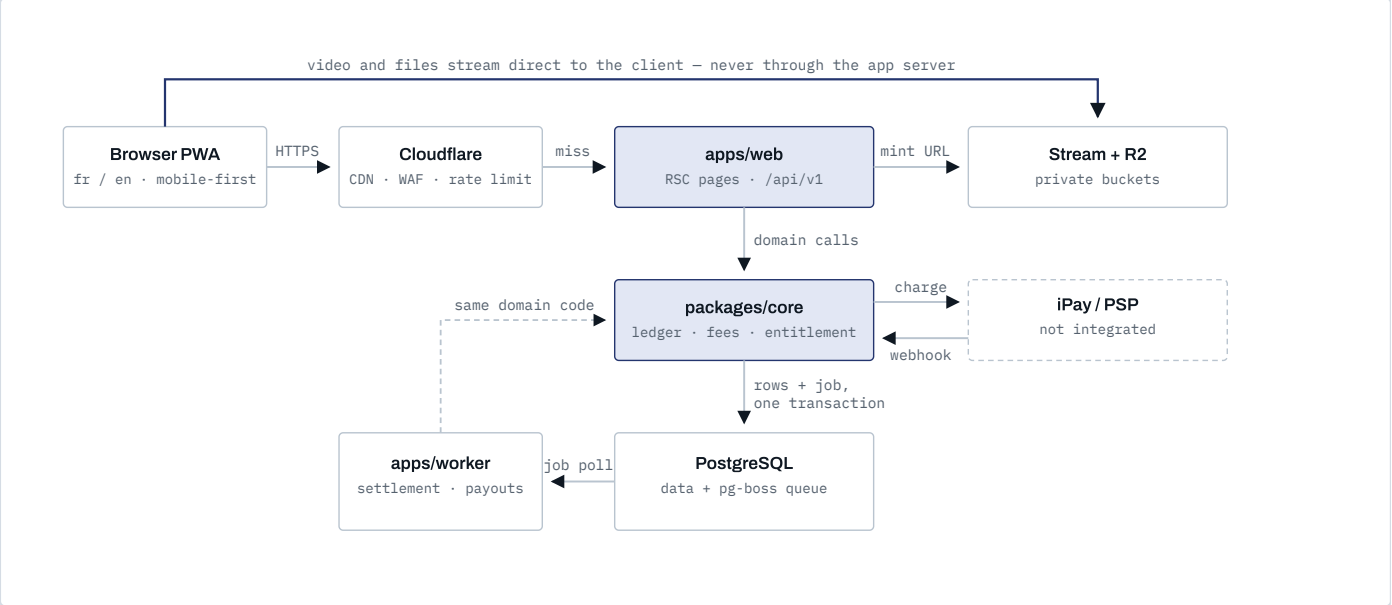
SWITCH IF

The team is junior-weighted or Prisma-experienced. In that case take Prisma and write the ledger's critical paths as raw SQL — the schema in section C is unchanged either way. Tell me the team composition and I will pick.

B System architecture

Four rules define the shape of the system. Everything else follows from them.

1. **Only `packages/core` touches the database.** Route handlers, server actions and job consumers all call the same domain services. There is no second path to the ledger.
2. **Jobs are enqueued in the transaction that creates their subject.** A settlement job and the ledger rows it will settle commit together or not at all.
3. **Media never passes through the application server.** The server mints a short-lived signed URL; the client fetches from storage directly.
4. **Webhooks are never processed inline.** Verify the signature, persist the raw event, return 200, and let the worker do the work. Payment providers retry aggressively and time out fast.



The database is the queue. Because pg-boss stores jobs in Postgres, a settlement job is inserted in the same transaction as the ledger entries it will later act on — the failure mode where money is recorded but its follow-up work is lost cannot occur. The dashed line from the worker means it imports `packages/core` rather than calling the web app.

Environments

Three: local (Docker Compose — Postgres, MinIO standing in for R2, a mock PSP), staging (real PSP sandbox, real Stream, anonymised data), production. Migrations run forward-only in CI with a tested rollback for each. Staging

must be able to run a full purchase-to-payout cycle before any production release touches the ledger.

C Database schema

Conventions that apply everywhere: UUIDv7 primary keys (time-sortable, so they index well and leak no sequence count); `timestamptz` in UTC only; status enums rather than soft deletes except where retention law requires otherwise; and **money is never a bare number** — `amount_minor bigint` always travels with a `currency char(3)`.

> **AFRICA-SPECIFIC CORRECTNESS TRAP**
>
> XOF and XAF have **zero** decimal places. NGN (kobo) and GHS (pesewa) have two. Any code that assumes "minor units means cents, divide by 100" will be wrong by a factor of 100 across the entire UEMOA zone. The exponent comes from the `currencies` table, never from a constant.

The financial core

This is the part I would write first and change least. The rest of the schema can evolve; this cannot.

```sql
-- Money is always a pair, and the exponent is data, not a constant.
CREATE TABLE currencies (
  code        char(3)  PRIMARY KEY,          -- XOF, XAF, NGN, GHS
  minor_unit  smallint NOT NULL CHECK (minor_unit BETWEEN 0 AND 4),
  is_active   boolean  NOT NULL DEFAULT true
);

CREATE TYPE ledger_account_kind AS ENUM (
  'user_available',  -- withdrawable
  'user_pending',    -- earned, not yet releasable
  'platform_escrow', -- captured, owed to somebody, not yet split
  'platform_revenue',
  'psp_clearing',    -- money at the payment provider
  'payout_payable'   -- approved withdrawal, not yet paid out
);

CREATE TABLE ledger_accounts (
  id          uuid PRIMARY KEY,
  kind        ledger_account_kind NOT NULL,
  owner_type  text,                        -- 'user' | 'store' | NULL for platform
  owner_id    uuid,
  currency    char(3) NOT NULL REFERENCES currencies(code),
  UNIQUE (kind, owner_type, owner_id, currency)
);

CREATE TABLE ledger_transactions (
  id              uuid PRIMARY KEY,          -- uuidv7
  kind            text NOT NULL,              -- capture | settlement | refund | payout …
  occurred_at     timestamptz NOT NULL DEFAULT now(),
  idempotency_key text UNIQUE,
  reverses_id     uuid REFERENCES ledger_transactions(id),
  metadata        jsonb NOT NULL DEFAULT '{}'   -- order_id, payment_id, actor…
);

CREATE TABLE ledger_entries (
  id             uuid PRIMARY KEY,
  transaction_id uuid    NOT NULL REFERENCES ledger_transactions(id),
  account_id     uuid    NOT NULL REFERENCES ledger_accounts(id),
  direction      smallint NOT NULL CHECK (direction IN (-1, 1)),
  amount_minor   bigint   NOT NULL CHECK (amount_minor > 0),
  currency       char(3)  NOT NULL REFERENCES currencies(code)
);

-- Append-only. The application role cannot rewrite history.
REVOKE UPDATE, DELETE ON ledger_entries, ledger_transactions FROM afrinext_app;

-- Every transaction balances, per currency. Deferred, so all entries
-- can be inserted before the invariant is checked at COMMIT.
CREATE CONSTRAINT TRIGGER ledger_must_balance
  AFTER INSERT ON ledger_entries
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION assert_transaction_balanced();

-- A cache, never the truth. Rebuilt and asserted nightly.
CREATE TABLE account_balances (
  account_id    uuid PRIMARY KEY REFERENCES ledger_accounts(id),
  balance_minor bigint NOT NULL DEFAULT 0,
  entry_count   bigint NOT NULL DEFAULT 0,
  updated_at    timestamptz NOT NULL DEFAULT now()
);
```

Entities by module

The brief listed candidate tables and asked me not to create them blindly. This is the filtered set, with the ones I would *not* build in phase 1 marked.

MODULE	TABLES	NOTES
Identity	`users`, `user_identities`, `sessions`, `profiles`, `roles`, `permissions`, `role_permissions`, `role_assignments`, `countries`, `currencies`, `locales`	`user_identities` holds one row per credential (phone, email, future OAuth) so a user can add a second login method without a schema change. `role_assignments` is scoped — see section D.
Stores	`stores`, `store_settings`, `store_payout_accounts`, **`STORE_MEMBERS`**	The *store* is the seller, not the user — orders reference `store_id`. `store_members` is deferred until multi-person stores are actually requested.
Catalog	`categories`, `products`, `product_media`, `digital_assets`, **`PRODUCT_VARIANTS`**, **`INVENTORY_ITEMS`**, **`SERVICE_OFFERINGS`**	One `products` table with a `kind` discriminator plus kind-specific side tables. Variants and inventory arrive with physical goods; service offerings with the services module.
Learn	`courses`, `course_modules`, `course_lessons`, `lesson_assets`, `course_enrollments`, `lesson_progress`, **`COURSE_CERTIFICATES`**	Certificates are a future feature in the brief and stay that way — but `enrollments` carries the completion data a certificate would need.
Commerce	`carts`, `cart_items`, `checkout_groups`, `orders`, `order_items`, `order_fee_lines`, `order_events`	`order_fee_lines` is the frozen commission snapshot — the reason changing a commission rate never rewrites history.
Payments	`payment_intents`, `payment_attempts`, `payment_events`, `refunds`	`payment_events` stores the raw provider payload with a `UNIQUE (provider, provider_event_id)` — this single constraint is what makes webhook replay harmless.
Ledger	`ledger_accounts`, `ledger_transactions`, `ledger_entries`, `account_balances`, `settlement_holds`, `commission_rules`, `idempotency_keys`	Above.
Payouts	`withdrawal_requests`, `payout_batches`, `kyc_records`	KYC tiers gate withdrawal ceilings. See section K.
Delivery	`delivery_partners`, `delivery_orders`, `delivery_events`, `delivery_proofs`, **`CASH_COLLECTIONS`**	Deferred to the physical-goods phase. `cash_collections` models the rider's cash float — see section I.
Referral	`referral_programs`, `referral_codes`, `referral_attributions`, `referral_commissions`	No `referral_relationships` tree — single level only, see section H.
Trust & ops	`reviews`, `notifications`, `audit_logs`, `platform_settings`, `feature_flags`, **`DISPUTES`**, **`DISPUTE_MESSAGES`**	Disputes arrive with physical goods, where they actually occur. Digital refunds in phase 1 are an admin action against an audited reversal.

Tables from the brief I would **not** create: `referral_relationships` (implied by single-level attribution), `user_roles` (superseded by scoped

`role_assignments`), and a separate `services` table (services are `products` with `kind = 'service'`).

D Authentication & permissions

Authentication

- **Phone + OTP is the primary method**, email + password secondary. Phone penetration far exceeds email use in the target countries, and phone is already the identifier for mobile money.
- **Database sessions, not JWTs.** An opaque token in an `httpOnly; Secure; SameSite=Lax` cookie, resolved against `sessions` on each request. A JWT cannot be revoked before it expires; a session that can move money must be killable in one query.
- Session rotation on privilege change, sign-in and password change. Shorter idle timeout for admin sessions than for shoppers.
- **Step-up re-verification** for money-sensitive actions: requesting a withdrawal, adding or changing a payout account, changing the phone number.

Authorization

Roles are scoped, additive rows. A single user simultaneously being a buyer, an instructor, a rider in Niamey and staff on someone else's store is the normal case, not an edge case.

```
CREATE TABLE role_assignments (
  id          uuid PRIMARY KEY,
  user_id     uuid NOT NULL REFERENCES users(id),
  role_id     uuid NOT NULL REFERENCES roles(id),
  scope_type  text NOT NULL,   -- 'global' | 'store' | 'course' | 'zone'
  scope_id    uuid,             -- NULL when scope_type = 'global'
  granted_by  uuid REFERENCES users(id),
  granted_at  timestamptz NOT NULL DEFAULT now(),
  UNIQUE (user_id, role_id, scope_type, scope_id)
);
```

Permissions are strings of the form `resource.action` — `product.publish`, `order.refund`, `withdrawal.approve`, `lesson.view`. Roles hold sets of them. Every check goes through one function:

```
// packages/core/authz — the only permission gate in the system.
await authorize(actor, 'product.publish', { type: 'store', id: storeId });
```

RULE

IDOR is prevented by **scoping the query**, not by fetching a row and then checking it. Repository methods take the actor and constrain in SQL: a seller's order list is `WHERE store_id = ANY(actor.storeIds)`, never `findById()` followed by an ownership test. The second pattern works until someone forgets it once.

ADMINISTRATIVE ROLES ARE SPLIT BY LEAST PRIVILEGE

ROLE	CAN	CANNOT
`support`	Read users, orders, enrollments; resend receipts; open disputes	Touch the ledger, approve payouts, change commission rules
`ops`	Moderate catalog, verify stores, manage categories and delivery zones	Anything financial
`finance`	Approve withdrawals, issue refunds, post adjustments, view the ledger	Edit catalog, change permissions, alter user identities
`superadmin`	Role and settings administration	Approve their own withdrawal — a hard, code-level self-approval block

E Financial architecture

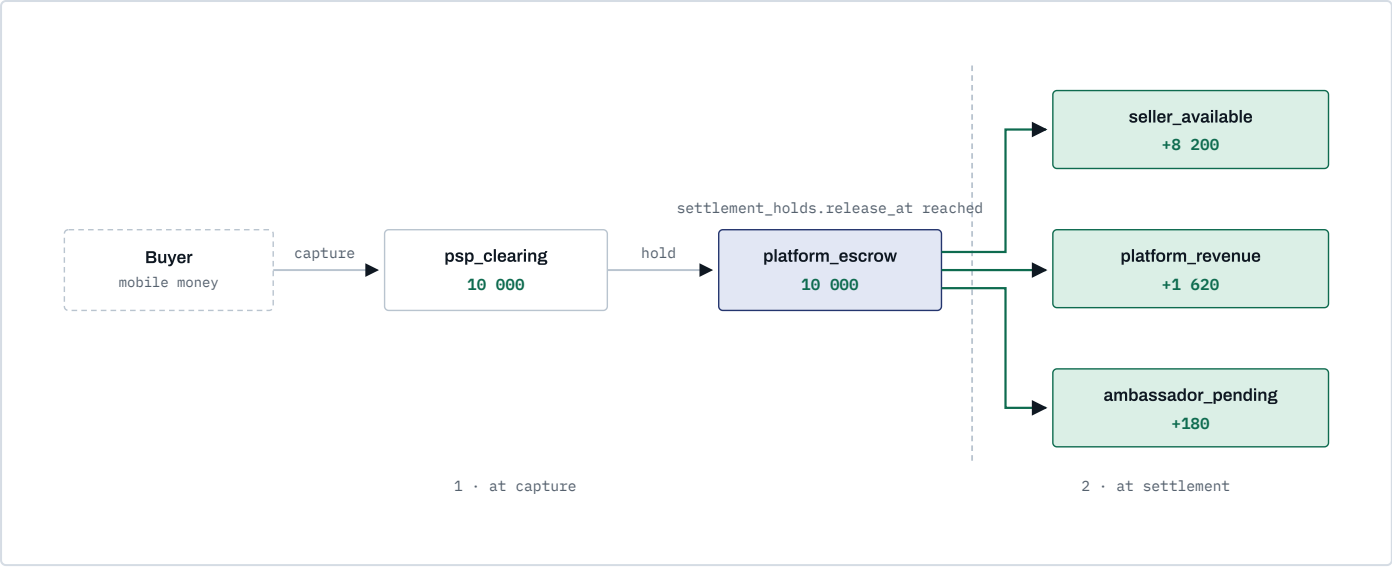
The wallet is not a balance field with rules around it. It is a **double-entry ledger**, and the balance is a derived quantity that we cache for speed and verify continuously.

The four invariants

1. **Entries are immutable.** No UPDATE, no DELETE — enforced by revoked grants, not by convention. A mistake is corrected by posting a compensating transaction linked through `reverses_id`.
2. **Every transaction balances.** Per transaction, per currency, the signed sum of entries is zero. A deferred constraint trigger rejects the COMMIT otherwise. Money cannot be created or destroyed by application code.
3. **Balances are cached, not authoritative.** `account_balances` is updated inside the same transaction under a row lock; a nightly job recomputes `SUM(direction × amount_minor)` per account and alarms on any drift.
4. **No floating point, anywhere.** `bigint` minor units and a currency code, from the database through the API to the rendered price.

Pending versus available

These are not statuses on a row. They are **two different accounts**, and "funds becoming withdrawable" is a transfer between them. That means the question "why is my money still pending?" is answerable by pointing at a specific ledger transaction and its `settlement_holds` row, rather than by reading application logic.



One 10 000 XOF course sale, at an 18% platform commission with a 10% referral program. Every arrow is one balanced ledger transaction. Note where the referral is drawn from: 180 is 10% of the *platform's* 1 800, not of the gross — so referral cost can never exceed what the platform earned on that sale. The ambassador's share lands in `pending` and is released separately once the buyer's refund window closes. For physical goods the seller's share also lands in `seller_pending` and moves to `available` on a second transaction when the delivery hold expires.

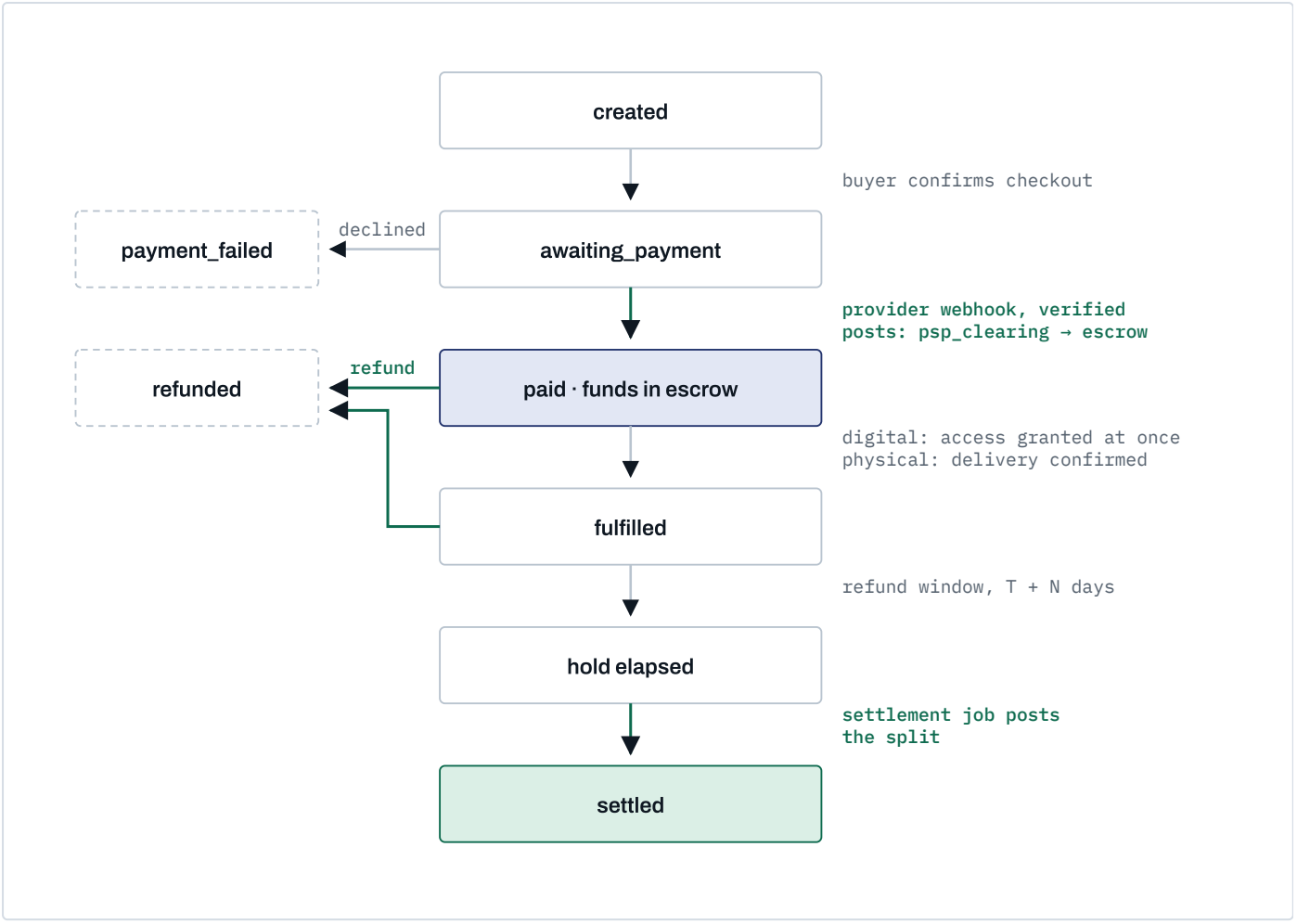
Rounding: the residual rule

Commission percentages rarely divide cleanly into whole francs. The rule is fixed and applied everywhere:

1. Compute every non-seller line (platform, referral, delivery) in minor units, rounding **half up**.
2. The seller's line is `gross − Σ(all other lines)`, never computed independently.

The split therefore re-sums to the gross by construction. No minor unit is ever created or lost to rounding, and the residual consistently favours the seller — a defensible position if it is ever questioned.

When money moves



Green transitions post ledger entries; grey ones only change order state. Money enters escrow when the provider webhook is verified — never when the buyer returns from a redirect. A dispute does not get its own state: it *extends* the hold by pushing `settlement_holds.release_at` forward, so disputed money simply never reaches `available` while the dispute is open.

The commission engine

Rules are data. Resolution happens once, at order creation, and the result is frozen.

```sql
CREATE TABLE commission_rules (
  id              uuid PRIMARY KEY,
  transaction_type text NOT NULL,   -- digital | course | physical | service | delivery | referral
  country_code    char(2),              -- NULL = any
  category_id     uuid,                 -- NULL = any
  store_id        uuid,                 -- NULL = any; a negotiated rate for one seller
  rate_bps        integer,              -- basis points; 1800 = 18.00%
  fixed_minor     bigint,
  currency        char(3) REFERENCES currencies(code),
  priority        integer NOT NULL,
  effective_from  timestamptz NOT NULL,
  effective_to    timestamptz
);
```

Resolution is most-specific-wins, broken by `priority` and then by `effective_from` — deterministic, and unit-testable in isolation. The resolved outcome is written to `order_fee_lines` with the `rule_id`, the basis and the computed amount.

> **WHY THE SNAPSHOT MATTERS**
>
> Raise the platform commission from 18% to 20% next quarter and **no historical order changes.** Every past settlement can still be re-derived from its own fee lines, and any seller asking "why did I receive this amount?" gets an answer that references the rule that was live at the moment they sold. A system that recomputes fees from current rules cannot answer that question truthfully.

Withdrawals

1. Request → validated against KYC tier ceiling, minimum amount, available balance, payout-account cooling period, and absence of open disputes.
2. Approved (automatically under a configured threshold, by `finance` above it) → ledger posts `user_available → payout_payable`. The money leaves the withdrawable balance at approval, so it cannot be double-spent while the payout is in flight.
3. Payout executed through the provider → on confirmation, `payout_payable → psp_clearing`.
4. On failure, a compensating transaction returns the amount to `user_available` and the request is marked failed with the provider's reason.

F Marketplace architecture

One products table, three kinds

`products.kind ∈ {physical, digital, service}`, with kind-specific side tables (`product_variants` + `inventory_items`, `digital_assets`, `service_offerings`). Not three separate tables — because carts, order items, reviews, search, favourites and fee resolution all need a single polymorphic target, and a three-table design forces a nullable triple foreign key into `order_items` that every query then has to unpack.

What differs between kinds is **fulfilment and hold policy**, and that lives in one place:

KIND	FULFILLED WHEN	DEFAULT HOLD
digital	Payment verified	24 h
course	Enrollment created	72 h
service	Buyer confirms, or auto after N days	7 d
physical	Delivery confirmed with proof	7 d

These are defaults in `platform_settings`, overridable per country and per category from the admin console — not constants in code.

Stores are the seller

Orders reference `store_id`, never `user_id`. This costs nothing now and buys three things later: a user can run more than one store; a store can gain staff without reshaping the order history; and a store can be transferred or suspended without touching the person's account, wallet or other roles.

Multi-store carts

A buyer will put a course from Dakar and a bag of shea butter from Bamako in one cart. At checkout the cart splits into **one order per store**, grouped under a `checkout_group_id`, paid by a single payment intent. Each order then settles independently on its own timeline — the course in 72 hours, the shea butter when it is delivered. Attempting to keep them as one order forces every fulfilment and refund decision to be all-or-nothing.

Search

Postgres full-text search with a generated `tsvector` column, plus `pg_trgm` for typo tolerance, and a French text-search configuration alongside English. Facets come from ordinary indexed columns. Move to Meilisearch or Typesense when the catalog passes roughly 100 000 rows or when faceted search gets heavy — not before, and not to Elasticsearch, which is an operational burden this team should not carry at launch.

Reviews

Only from a verified purchase: `reviews.order_item_id` is a required foreign key, unique per buyer per item. This kills fake-review farming structurally rather than through moderation, which matters when reviews feed seller ranking and seller ranking feeds income.

G Afrinext Learn & video access

Structure is conventional — `courses → course_modules → course_lessons → lesson_assets`, with lesson kinds `video | pdf | text`. The interesting parts are entitlement and bandwidth.

Entitlement is checked at token-mint time, on every request

1. The player asks `GET /api/v1/learn/lessons/:id/playback`.
2. The server resolves the actor, checks `course_enrollments` for a live entitlement (and `access_expires_at` if the course is time-limited), and calls `authorize(actor, 'lesson.view', {course})`.
3. Only then does it mint a **signed Stream token, scoped to that user, valid 2–5 minutes**, and return an HLS manifest URL.
4. The player refreshes the token mid-playback as it nears expiry.
5. Every mint is written to `audit_logs` — which also gives instructors real "who watched what" analytics for free.

> **NEVER**
>
> Paid course content never has a permanent public URL. Not in a public bucket, not behind an unguessable path, not with a long-lived signed URL. An unguessable URL is a password that gets forwarded on WhatsApp.

Downloads are a per-asset instructor choice

`lesson_assets.downloadable boolean`, set by the instructor per asset. When true, downloading issues a separate short-TTL signed URL, logged, and rate-limited per user. When false, no download URL is ever minted — the flag is enforced server-side at mint time, not by hiding a button.

What is realistic about protection

> **BE HONEST WITH INSTRUCTORS**
>
> Signed short-lived URLs plus a burned-in overlay carrying the viewer's phone number make casual re-sharing traceable and inconvenient. They do **not** prevent a determined screen recording. Genuine prevention requires Widevine/FairPlay DRM, which is expensive and out of scope here. Instructors should be told this plainly rather than sold an assurance the platform cannot honour.

Progress and resume

`lesson_progress(enrollment_id, lesson_id, position_seconds, furthest_seconds, completed_at)`, written by a heartbeat every ~15 seconds, throttled, and flushed on pause and on unload via `sendBeacon`. `course_enrollments.last_lesson_id` gives cross-device resume: the learner picks up on their phone exactly where they stopped on a shared laptop. Keeping `furthest_seconds` separate from `position_seconds` means scrubbing backwards does not destroy completion state.

Built for the actual network

- An encoding ladder that **starts at 240p**, not at 720p. Adaptive bitrate matters far more here than peak quality.
- An audio-only rendition for lecture-style courses, selectable by the learner to save data.
- Progress writes are a few dozen bytes and batched — a learner should never lose their place because the heartbeat could not get through.
- Where the instructor permits downloads, offline viewing is the difference between a finished course and an abandoned one.

H Afrinext Network (referrals)

The brief's principle — pay for real economic activity, never for recruitment — is enforced structurally here, not by policy text.

Attribution

- An ambassador gets a code; the link is `/r/:code`, which sets a first-party cookie (90 days) and redirects.
- At signup the cookie is consumed into `referral_attributions(referrer_user_id, referred_user_id, program_id, attributed_at, expires_at)`, with `UNIQUE(referred_user_id)`.
- **First touch, immutable.** The first ambassador to introduce someone keeps the attribution. Last-touch invites a scramble to overwrite each other's referrals just before a purchase.

Qualification

A commission is created only when a **settled ledger transaction** of a configured type exists — not at signup, not on profile completion, not on "becoming active". There is no code path that pays for a registration, because commissions are computed by a job that reads settled transactions and has no other input.

CONFIGURABLE PER PROGRAM	EXAMPLE
Basis	`platform_revenue`
Rate	`1000 bps (10%)`
Duration from attribution	`365 days`
Per-referral lifetime cap	`150 000 XOF`
Eligible transaction types	`course, digital, physical, service`
Country scope	`NE, ML, BF …`

CONFIRMED DECISION

Single level only — no downline of a downline

WHY	**Confirmed by the business: A earns from B's qualifying activity; A never earns from B → C.** Multi-level structures are where recruitment-scheme regulatory attention concentrates, and they contradict Afrinext's own stated principle. One level keeps the model explainable to a regulator in one sentence: "we share part of our commission with whoever introduced the seller."
INSTEAD OF	Two or more tiers, which measurably increases ambassador enthusiasm and measurably increases legal exposure.
TRADE-OFF	Slower ambassador-network growth than a tiered scheme would produce.
NOTE	Because the commission basis is platform revenue, the payout is bounded by definition — the platform can never owe more in referral than it earned on the sale. That guardrail is what makes the program safe to run at all, and it disappears if the basis is ever changed to gross.

Fraud controls

- Self-referral detection across phone number, device fingerprint, payout account and payment instrument.
- Referral commissions land in `ambassador_pending` and release only after the underlying order clears its refund window — so a wash sale that is refunded never pays out.
- Velocity limits per ambassador, with automatic hold and review above configured thresholds.
- Circular attribution (A refers B refers A) rejected at write time.

I Delivery architecture

Deferred to the physical-goods phase, but designed now because it constrains the order state machine.

STATE MACHINE

`requested → offered → accepted → picked_up → in_transit → delivered`, with `failed` and `cancelled` as terminal branches. Every transition writes a `delivery_events` row with actor, timestamp and GPS point — the audit trail is the record, the current status is a cached projection of it.

DISPATCH

Offers broadcast to eligible partners in the zone; first acceptance wins via a conditional update (`UPDATE … WHERE status = 'offered'`) so two riders can never both hold the same job. Unaccepted offers expire and re-broadcast with a widening radius.

PROOF OF DELIVERY

A short OTP shown in the buyer's app and entered by the rider, plus an optional photo and the GPS fix at completion. The OTP is what flips the order to `fulfilled` and starts the settlement hold — a rider cannot self-declare a delivery into existence.

RIDERS LOSE SIGNAL CONSTANTLY

The rider interface queues state transitions locally and replays them with idempotency keys when connectivity returns. A transition applied twice must be a no-op; a transition applied out of order must be rejected by the state machine, not silently accepted.

> **CASH ON DELIVERY IS A BIGGER PROBLEM THAN IT LOOKS**
>
> COD is the dominant payment method for physical goods across much of the region, and it inverts the money flow: the rider collects cash, so the platform is owed money by the rider rather than holding the buyer's funds. That needs a `cash_collections` liability account per rider, a deposit and reconciliation process, float limits, and a real operational routine — it is a project, not a checkbox. **My recommendation is to launch physical goods prepaid-only** and add COD as its own phase with its own design.

J API architecture

Two entry styles, one implementation. **Server Actions** for first-party form mutations (fewer round trips, progressive enhancement) and **versioned REST** at `/api/v1/*` for everything a future native app, an ambassador tool or a partner integration would consume. Both call the same `packages/core` services. Logic is never duplicated between them, and neither one is allowed a private path to the database.

DOMAIN	REPRESENTATIVE ENDPOINTS
auth	POST /auth/otp/request · /auth/otp/verify · /auth/session · DELETE /auth/session
me	GET /me · PATCH /me · GET /me/roles · GET /me/orders · GET /me/enrollments
stores	POST /stores · GET /stores/:slug · PATCH /stores/:id · GET /stores/:id/analytics
catalog	GET /products · GET /products/:slug · POST /stores/:id/products · POST /products/:id/publish
search	GET /search?q&kind&category&country&cursor
cart · checkout	GET/POST /cart/items · POST /checkout · GET /checkout/:groupId
orders	GET /orders · GET /orders/:id · POST /orders/:id/cancel · POST /orders/:id/refund
payments	POST /payments/intents · GET /payments/:id · POST /webhooks/:provider
wallet	GET /wallet · GET /wallet/transactions · POST /wallet/withdrawals · GET /wallet/withdrawals/:id
learn	GET /courses/:slug · GET /learn/lessons/:id · POST /learn/lessons/:id/progress · GET /learn/lessons/:id/playback
delivery	GET /delivery/offers · POST /delivery/:id/accept · POST /delivery/:id/transition
referrals	GET /referrals/me · GET /referrals/commissions · GET /r/:code
admin	/admin/* — mirrors the above under `admin.*` permissions

CONVENTIONS

- `Idempotency-Key` **is required** on every POST that moves money. The key, the request hash and the resulting transaction id are stored; a replay returns the original response rather than acting twice.
- Cursor pagination everywhere (`?cursor=&limit=`). Offset pagination on a growing catalog silently skips and duplicates rows.
- Responses are `{ data, meta }`; errors are `{ error: { code, message, fields } }` with **stable machine-readable codes** — clients must never branch on message text, which is localised.
- Money crosses the wire as `{ amountMinor: 8200, currency: "XOF" }`. Never a formatted string, never a float.
- Rate limits by route class: strictest on OTP send and verify, then auth, then write, then read.
- Webhooks return `200` as fast as possible after signature verification and persistence. All work happens in the worker.

K Security architecture

CONTROL	WHERE IT IS ENFORCED
Authentication	Opaque session tokens in `httpOnly; Secure; SameSite=Lax` cookies, resolved server-side. Rotated on sign-in and privilege change. Argon2id for the password path.
Step-up verification	Fresh OTP required for withdrawal requests, payout-account changes and phone changes — regardless of session age.
Authorization	One `authorize()` choke point in `packages/core/authz`. No route handler makes its own decision.
IDOR	Prevented by scoping queries to the actor in SQL, not by post-fetch ownership checks.
Input validation	Zod at every boundary; database CHECK and FK constraints as the last line. Frontend validation is a convenience and is never trusted.
Rate limiting	Per IP and per user, at the edge and in the app. Hardest limits on OTP send/verify, login, withdrawal, and playback-token minting.
Webhook integrity	Signature verified against the **raw** body before parsing; timestamp freshness window; `UNIQUE (provider, provider_event_id)` makes replay a no-op.
File access	Private buckets only. Short-TTL signed URLs minted after an entitlement check. Every mint audited.
Payment card data	Never touched. Provider-hosted collection only, which keeps PCI scope at SAQ-A. This is a deliberate architectural constraint, not an oversight.
Audit	Append-only `audit_logs` for every admin action and every financial state change: actor, action, target, before/after, IP, user agent, request id.
Fraud	Velocity checks, device fingerprinting, a cooling period before first withdrawal to a newly added payout account, and automatic hold-and-review thresholds.
Separation of duty	No user can approve a withdrawal to an account they control. Enforced in code, not in policy.
Secrets	Environment only, distinct per environment, rotated. Sandbox and production provider credentials never coexist in one config.
Data protection	PII minimised and encrypted at rest; retention policy per data class; export and deletion paths built for Nigeria's NDPR, Ghana's Data Protection Act and UEMOA national regimes.

EXPLICITLY OUT OF SCOPE, AND NAMED AS SUCH

DRM for video (Widevine/FairPlay). A third-party penetration test — which should be scheduled and budgeted *before* public launch, not after. Formal PCI-DSS certification beyond SAQ-A. Automated AML transaction-monitoring beyond threshold rules.

L Project structure

pnpm workspaces plus Turborepo. The directory layout is not cosmetic — it is what physically prevents domain logic from leaking into the framework.

```
afrinext/
├─ apps/
│  ├─ web/                 Next.js 16 — pages, route handlers, server actions
│  └─ worker/              long-running pg-boss consumers
├─ packages/
│  ├─ core/                framework-free domain logic — the application
│  │  ├─ money/            Money type, minor units, rounding, formatting
│  │  ├─ ledger/           posting, balances, reconciliation
│  │  ├─ commissions/      rule resolution, fee snapshots
│  │  ├─ orders/           order + settlement state machines
│  │  ├─ payments/         PaymentProvider interface + registry
│  │  ├─ learn/            entitlement, playback tokens, progress
│  │  ├─ delivery/
│  │  ├─ referrals/
│  │  └─ authz/            permissions, authorize()
│  ├─ db/                  Drizzle schema, SQL migrations, seeds
│  ├─ providers/           psp-ipay, psp-manual, storage-r2, video-stream, sms, email
│  ├─ ui/                  design system (seeded from today's prototype)
│  ├─ i18n/                fr / en message catalogs, shared with the worker
│  └─ config/              tsconfig, eslint, tailwind preset
├─ docs/
│  ├─ architecture/        this document
│  ├─ adr/                 architecture decision records, numbered
│  └─ runbooks/            reconciliation drift, stuck payout, failed webhook
└─ infra/                  docker, compose, CI, deploy
```

THE RULE THAT MAKES THIS WORTH DOING

`packages/core` may import `packages/db`. It may **not** import anything from `next`, and this is enforced by an ESLint boundary rule in CI, not by discipline. The consequence: the ledger and commission engine are testable in milliseconds without a server, the worker runs identical business logic to the web app, and extracting a standalone API service later is a packaging change rather than a rewrite.

M Development roadmap

Phases are ordered by *irreversibility*, not by visibility. The money spine comes before any product feature, because it is the thing that cannot be retrofitted. Durations assume two to three engineers and are estimates, not commitments.

PHASE	SCOPE	EXIT CRITERION — THE THING THAT MUST DEMONSTRABLY WORK	EST.
P0 Foundations	Monorepo, Postgres, Drizzle schema v1, auth (phone OTP + email), scoped RBAC, i18n, design system extraction, CI, observability skeleton	A user registers by phone, signs in, switches language. CI runs typecheck, lint, tests and migrations forward and back on every commit.	2 wk
P1 Money spine	`money`, `ledger`, commission engine, idempotency, settlement holds, reconciliation job, admin ledger viewer	Property-based tests over 10 000 randomised operations prove every transaction balances and the cached balance equals the sum of entries. An injected drift is caught by the reconciliation job. **No product feature is built before this passes.**	3 wk
P2 Stores & catalog	Store creation, digital products and courses as products, categories, media upload to R2, public storefront and product pages, search	A seller publishes a paid digital product that renders server-side at a public URL with correct metadata and appears in search.	3 wk
P3 Cycle closed, no PSP	Cart, checkout, orders, fee snapshots, and a `ManualTransferProvider` where an admin confirms receipt	The complete economic cycle runs end to end without any payment integration: buyer orders → admin confirms → ledger settles → seller sees earnings → requests withdrawal → finance marks it paid. **This de-risks everything that depends on iPay.**	3 wk
P4 iPay	Real provider implementation behind the existing interface	In sandbox: charge created, webhook signature verified, status query reconciled, refund executed, replay proven harmless. GATED Cannot start until the documentation and sandbox credentials in section P are in hand.	unknown
P5 Afrinext Learn	Modules, lessons, Stream upload, entitlement, signed playback, progress and resume, download flag, learner interface	An automated test proves a paid lesson is unplayable without entitlement, and that resume works across two devices for one learner.	4 wk
P6 Wallet & payouts	Wallet interface, KYC tiers, withdrawal flow with step-up auth, payout batches, finance console	Full withdrawal lifecycle including a provider failure that correctly reverses back to available balance.	3 wk
P7 Network	Referral programs, attribution, qualification, commissions, ambassador dashboard, fraud checks	Tests prove a commission can only originate from a settled transaction, that self-referral is blocked, and that a refunded order pays nothing.	3 wk
P8 Physical & delivery	Variants, inventory, shipping, delivery partner interface, state machine, proof of delivery, delivery earnings	A physical order settles only on OTP-confirmed delivery, and a rider replaying a queued transition twice changes nothing.	5 wk
P9 Services & creators	Service offerings, booking and request flow, creator categories and portfolios	A service settles on buyer confirmation, and on auto-confirmation after the configured window.	3 wk
P10 Hardening	Load test, penetration test, runbooks, restore drill, compliance review, admin settings completeness	A database restore is performed from backup against a stopwatch, and the recovery time is written down.	3 wk

The admin console is **not** a phase. Each phase ships the admin surface for what it built; an admin console deferred to the end is an admin console that never gets finished.

To the recommended launch vertical (P0–P3, P5, P6, plus P4 whenever iPay unblocks): roughly **18 weeks**. Full scope as briefed: roughly **32 weeks**. If iPay stalls, P3's manual rail means you can onboard real sellers and take real money by bank transfer while it is resolved.

R Risk register

RISK 1 — THE ONE THAT CAN STOP THE PROJECT

Afrinext receiving customer money and paying sellers may be a regulated activity. The operator, AFRI NEXT TECHNOLOGIE, is a registered *Entreprise Individuelle* with an NIF and RCCM — which establishes layer A in section 01, and nothing beyond it. In the UEMOA zone, taking money from a buyer, holding a seller's entitlement and disbursing it can fall under BCEAO e-money and payment-services rules. Afrinext intends to investigate and obtain whatever authorization is required. Until that resolves, the architecture reduces exposure the only way engineering can: the ledger records *entitlement* rather than asserting custody, and the payout rail sits behind an interface so the answer can change without a rewrite. **This gates P6 (payouts), not P0 or P1.**

RISK	SEV	MITIGATION
Ledger correctness — a bug here is money, and it is discovered by users	HIGH	Double-entry with database-enforced balancing, immutability by revoked grants, property-based tests, nightly reconciliation with alarms, ledger built in P1 before anything depends on it
iPay is an unknown — the API may lack payouts or refunds entirely	HIGH	Provider abstraction plus the manual rail in P3, so no other phase is blocked. Section P lists exactly what is needed to remove this risk
Referral fraud and scheme perception	MED	Single level, commission drawn from platform revenue only, qualification from settled transactions only, pending-until-refund-window, self-referral detection
Scope — the brief is a 32-week product and enthusiasm compounds	HIGH	The launch vertical in section 00; phases with hard exit criteria; nothing starts until the prior phase's criterion demonstrably passes
KYC and AML on withdrawals	MED	Tiered KYC with withdrawal ceilings per tier; identity verification before the first payout; threshold-based holds. Provider choice still open
Mobile money reversal semantics differ from cards and vary by operator	MED	Conservative hold windows, configurable per country and per provider; never assume card-like chargeback behaviour
Video bandwidth cost and learner data cost	MED	R2 zero-egress, ladder starting at 240p, audio-only rendition, permitted offline download
Latency from West Africa	LOW	Postgres in `eu-west-3`, Cloudflare edge caching. Measure real round-trip from Niamey, Lagos and Accra in P0 and revisit if the numbers disagree with the assumption
Cash on delivery reconciliation	MED	Recommend launching physical goods prepaid-only; COD gets its own phase and its own liability model
Cross-jurisdiction data protection	MED	PII minimisation, encryption at rest, per-class retention, export and deletion paths built in P0 rather than bolted on
Operating a money system on-call	MED	Runbooks written in the phase that creates the failure mode; reconciliation alarms route to a human; restore drill rehearsed in P10

P iPayMoney: confirmed provider, unverified API

iPayMoney is the confirmed initial payment provider, and Afrinext already holds a merchant account. That settles the commercial question. It does not settle the engineering one.

> **VERIFICATION STATUS — READ BEFORE TRUSTING ANYTHING BELOW**
>
> The official iPayMoney documentation is **not present in this repository**, and the provider's domain is **not reachable from the build environment** — outbound egress to it is blocked by the network policy. I have therefore verified *nothing* about their API: not the authentication scheme, not an endpoint, not a payload, not a webhook signature, not a status value.
>
> The capabilities reported to me — Mobile Money, credit cards, local and international cards, a JavaScript SDK and an Android SDK — are recorded here as **stated by the business, pending confirmation from the official documentation.** No code treats them as facts.

ITEM	STATE
`PaymentProvider` interface	**BUILT** Provider-agnostic; nothing in it assumes iPayMoney.
`MockPaymentProvider`	**BUILT** Deterministic, in-process, for development and tests. Refuses to load when `NODE_ENV=production`.
`iPayMoneyProvider` adapter	**BOUNDARY ONLY** The adapter and its registration exist; every method throws `ProviderNotConfiguredError` naming the missing documentation. It cannot silently appear to work.
Real endpoints, auth, payloads, webhook verification	**NOT IMPLEMENTED** Blocked on the list below.
Integration test structure	**PREPARED** Specs exist and skip unless sandbox credentials are present in the environment.

The abstraction Afrinext codes against. `createPayout` is optional deliberately — if iPayMoney has no disbursement API, payouts run as an operational batch and the platform still works, but the finance team's workload changes substantially, so it is worth establishing early.

```ts
interface PaymentProvider {
  readonly id: string;
  createCharge(input: ChargeInput): Promise<ChargeResult>;
  getCharge(providerRef: string): Promise<ChargeStatus>;
  verifyWebhook(rawBody: Buffer, headers: HeadersLike): Promise<VerifiedEvent>;
  refund(input: RefundInput): Promise<RefundResult>;
  createPayout?(input: PayoutInput): Promise<PayoutResult>;   // may not exist
  getPayout?(providerRef: string): Promise<PayoutStatus>;
}
```

Exactly what is needed to unblock the adapter

Items 6, 7 and 8 are the ones that most often invalidate an assumed design, so they matter most.

#	REQUIRED FROM THE OFFICIAL DOCUMENTATION	WHY IT BLOCKS
1	Base URLs, sandbox and production	Nothing can be called without them
2	Authentication scheme (API key, HMAC, OAuth) and where credentials go in a request	Determines secret storage and rotation
3	Charge initiation request/response schema, and supported channels per country	Determines the checkout data model
4	Flow type: hosted redirect, inline widget, JS SDK, or USSD push	Changes the checkout interface, not just the backend
5	Status query endpoint and the *complete* status enumeration	Reconciliation cannot be written against a partial enum
6	Webhooks: registration, payload schema, signature algorithm and header name, timestamp tolerance, retry policy	Money enters the ledger on a verified webhook; with no signature scheme there is no safe way to accept one
7	Currency handling — specifically whether XOF is sent as whole francs	XOF has zero decimal places; a wrong assumption is a 100× error
8	Whether a payout/disbursement API exists, and its settlement timing	Decides whether payouts are automated or operational
9	Refund API and its constraints (partial refunds, time limits)	The refund reversal path in the ledger
10	Idempotency support on charge creation, and rate limits	Retry safety
11	Settlement schedule to the merchant account (T+N) and fee structure	Hold windows and platform margin
12	Sandbox credentials and test numbers or accounts per channel	Nothing can be tested without them
13	JavaScript and Android SDK documentation, if the SDK route is used	Changes the client integration entirely

Drop the documentation into `docs/providers/ipaymoney/` in any form — PDF, exported HTML, an OpenAPI file, a Postman collection. Once it is there the

adapter is implementable against verified facts instead of assumptions.

S Status register

Anything unimplemented, unverified or assumed is listed here rather than left to inference. Current as of revision 2 of this document.

ITEM	STATUS
Sections A–M of this blueprint	`ARCHITECTURE` Design, not code. Phase 0 implements the foundation only; the rest is not built.
iPayMoney integration	`NOT IMPLEMENTED` Merchant account exists; the API is unverified and unreachable from the build environment. Interface and mock exist; the adapter throws. See section P.
Regulatory authorization (layer G)	`UNRESOLVED` Company registration (NIF, RCCM) establishes layer A only. Requires local counsel and, if needed, an application to the competent authority. Gates P6.
Legal document texts (Terms, Privacy, Seller, Instructor, Payout, Referral)	`NOT WRITTEN` The consent *architecture* exists; the documents themselves are a legal deliverable, not an engineering one.
Cloudflare Stream / R2, SMS provider, KYC provider	`NOT SELECTED` Recommended, not procured. No accounts, no confirmed pricing.
Latency assumption (`eu-west-3` over `af-south-1`)	`UNVERIFIED` Based on general routing, not measurement from Niamey. Verify in P0 or P1.
Effort estimates in section M	`ESTIMATES` Two to three engineers assumed. Not commitments.
ORM choice (Drizzle vs Prisma)	`PROCEEDING ON THE RECOMMENDATION` Phase 0 uses Drizzle. The decision genuinely depends on team composition, which is still unanswered — see section Q.
V2 (physical, delivery) and V3 (creator, services) architecture	`DESIGNED, NOT BUILT` Retained deliberately.

Q Decisions: settled, and still open

Settled — reflected throughout this document

DECISION	OUTCOME
Operating company	AFRI NEXT TECHNOLOGIE, *Entreprise Individuelle*, Niger, NIF + RCCM. Registration is layer A and nothing more — section 01.
Economic model	Afrinext is the commercial platform of record: it receives customer payments and pays beneficiaries under its own terms.
Launch scope	V1 = digital products and courses. V2 physical + delivery, V3 creator + services, both designed and deferred.
Launch market	Niger, XOF — as data, not as constants.
Payment provider	iPayMoney, merchant account held. Behind the provider interface; a second provider must remain possible.
Referral	Single level. Basis is Afrinext's realized commission, never gross. No payment for registration or recruitment.
Ledger	Double-entry design approved in principle. Not to be replaced by a balance field.
Consent	Versioned acceptance per document, per user. Establishes contract, not authorization.

Still open

None of these block Phase 0. Items 1 and 2 block later phases and are worth starting now because they have long lead times.

#	QUESTION	BLOCKS
1	**Regulatory.** Who is advising on layer G, and what authorization is Afrinext pursuing — its own licence, or operating under a licensed institution?	P6 payouts. Long lead time — start now.
2	**iPayMoney documentation.** The 13 items in section P. Sandbox credentials are the single most useful thing you can send.	P4 payment integration
3	**Team composition.** How many engineers, and what is their Prisma versus raw-SQL experience?	Settles the ORM choice; Phase 0 proceeds on Drizzle meanwhile
4	**Hosting.** Managed (Vercel + Neon, higher unit cost) or containers (Hetzner or Fly, cheaper, more operations)?	P0 deployment, not P0 code
5	**SMS provider for OTP in Niger.** Which aggregator, and what are the per-message economics?	Real OTP delivery. The OTP architecture is provider-agnostic, so this is not urgent.
6	**Native app.** Is the PWA sufficient at launch, or is store distribution required?	How much weight the REST layer carries
7	**KYC.** Which identity documents are practically verifiable in Niger, and is there an existing provider?	P6 withdrawal tiers
8	**Legal document texts.** Who drafts the Terms, Seller Terms, Payout Terms and the rest?	Nothing technical — the consent machinery works with any text

---

Phase 0 is authorised and in progress: the monorepo, the schema, authentication, scoped permissions, the consent tables, the audit log, the ledger with its property tests, and the payment-provider abstraction. Phase 1 does not begin without an explicit approval.